

COM668 Computing Project

Assessment Task 4 – Project Review Report

Project title: Helpdesk application

Student name: Joseph McCusker

Student ID number: B00885401

PSG identifier: PSG-F08

Mentor name: Billal Ahmed Lodhi

Course title: Computing Technologies

Year of submission: 2025/2026

Word Count (2400 Maximum): 2352

You are required to self-declare the word count in this report.

A falsified word count may be considered as academic misconduct.

This is in keeping with Ulster University policy.

The Module Handbook and guidance has advice regarding what must be included in the word count.

Declaration:

I declare that this is all my own work. Any material I have referred to has been accurately referenced and any contribution of Artificial Intelligence technology has been fully acknowledged. I understand the importance of academic integrity and have read and understood the University's General Regulation: Student Academic Integrity and the Academic Misconduct Procedure. I understand that I must not upload my work before, during or after submission to any unapproved plagiarism detectors or answer-sharing platforms, or equivalent, and that only University-approved platforms should be used.

Contents

1. Software realisation	1
1.1. Implementation review.....	2
1.2. Review of development tools used	2
2. Quality assurance.....	4
2.1. Description of verification and validation activities.....	4
2.2. Testing.....	5
3. Critical appraisal	6
3.1. Evaluation of project outcomes	6
3.2. Review of plans and any decisions	6
3.3. Evaluation of methodology	7
3.4. Personal development.....	7
References	8
Appendix A - Code Manifest	8
Additional appendices.....	10

1. Software realisation

(suggested word count for Software realisation: 600)

1.1. Implementation review

My helpdesk application is built as a client-server system with Fast API backend and plain HTML, CSS and JavaScript frontend. The backend exposes a REST API for authentication, ticket management, user administration, notifications, and the frontend uses fetch to consume these endpoints.

Authentication and Authorisation:

Authentication uses JWT. Users log in with email and password; the backend verifies credentials and returns a signed token. Subsequent requests send the token in the Authorisation header, and the backend validates it and resolves the current user from the database. Passwords are hashed with bcrypt and never stored in plain text. Access control is role-based: end users, support staff and admin all have different capabilities. End users can create their own ticket statuses to see whether it's resolved or in progress, and admins can manage user roles and their statuses in the system.

Ticket Management:

Tickets follow a defined lifecycle: open > in progress > resolved. Creation stores the creator's ID, timestamps, and an initial status. Listing is filtered by role: end users see only their tickets, while administrators see all in the system. Status changes and other edits are restricted to administrators and are recorded in a ticket_history collection to support audits and traceability.

Notifications and communications:

When an administrator marks a ticket as resolved, the system sends a notification to the ticket creator, usually as an in-app notification or an email via SendGrid. End users see these in-app notifications the next time they log in after their ticket has been declared as resolved. This meets the requirement to notify users that their IT issue has been resolved, which was a key feature of my project proposal.

Frontend architecture:

The frontend is a single-page application with a login view and a main page dashboard view. After logging in, the UI switches to a dashboard with a sidebar (admin-only items such as user management), a ticket creation form, and a ticket list. The UI adapts based on the role of the person logging in to the system, such as showing admin sections for known admins or displaying the notifications panel with unread 'ticket resolved' messages and a dismiss action. Layout and styling are handled in a separate CSS file. In the original briefing, my plan was to implement the frontend using Angular, but I was unsuccessful and opted for HTML and CSS as a simpler, more familiar option for the application's development.

Data and persistence:

MongoDB is used as the primary store, with collections for users, tickets, ticket_history and notifications. The backend uses Motor for asynchronous database access, avoiding blocking under load. Configuration (e.g MongoDB URL, secret keys) is externalised via environment variables and .env files.

Strengths:

The key strengths of the application include a clear separation of concerns (routers by feature), role-based access control, audit history, and dual notification channels (in app and email). The system can be tested without SendGrid by using console logging fallback.

1.2. Review of development tools used

Python and FastAPI:

The backend is implemented in Python using the Fast API framework. Fast API provides automatic request/response validation, Open-API documentation, and native async support. Pydantic models define data schemas for users, tickets, and notifications, so invalid payloads are rejected before reaching application logic. Fast-API's dependency injection (e.g. Depends 'get_current_active_user') keeps authentication and authorisation logic centralised and reusable.

MongoDB and Motor:

MongoDB is used for persistence, with Motor as the async driver. The document model fits flexible structures such as ticket metadata, history entries, and notifications. Motor's async API aligns with Fast-API's async handlers, enabling concurrent request handling without blocking. Queries use simple filters, and indexes can be added later if needed.

Authentication Libraries:

The project uses 'python-jose' for JWT creation and verification and 'passlib' with bcrypt for password hashing. These are standard libraries for secure authentication in Python web applications.

Frontend tooling:

The frontend is built with plain HTML, CSS, and JavaScript. There is no build step or framework; the app is served by Python's 'http.server' module during development. This keeps dependencies minimal and the stack easy to understand and run. For production, a static file server or CDN would typically be used.

Email delivery:

SendGrid is integrated for transactional email. When 'sendgrid_api_key' is set, emails are sent via the SendGrid API; when it is not set, the application logs what would be sent. This supports local development and testing without email configuration.

Development environment:

Python virtual environments (venv) isolate project dependencies. Configuration is managed through .env and 'python-dotenv', so sensitive values stay out of source control. The backend runs with 'Uvicorn' in reload mode for iterative development.

Overall Assessment:

The chosen tools are appropriate for the scope: FastAPI for a clear, maintainable API; MongoDB for flexible data; JWT for stateless auth; and a simple frontend for a focused demo. The stack is coherent and supports good separation between API, business logic, and persistence.

2. Quality assurance

(suggested word count for Quality assurance: 800)

2.1. Description of verification and validation activities

Verification was done in the project by checking that each major requirement had a traceable implementation:

- Functional requirements: User registration and login, role – based access (end user, support staff, administrator), ticket creation and listing, administrator-only ticket status updates, user management for administrators, dashboard statistics, email notifications on status change (when SendGrid is configured), and in-app notifications when a ticket is resolved.
- Technical requirements: REST API with consistent endpoints, JWT authentication, MongoDB persistence, and a web frontend that calls the API.

Validation was completed by walking through basic scenarios e.g., an end user logs an IT issue; an administrator changes the ticket status to resolved; the end user logs in again and sees the in-app notification. That confirms the workflow matches the intended helpdesk use case.

Activities used:

1. Requirements traceability: Each feature was checked against the brief (e.g., only admins resolve tickets or end users are notified when tickets are resolved). This is informal traceability- mapping bullet points to routes and UI behaviour.
2. API checks: FastAPI's automatic OpenAPI documentation (/docs) was used to confirm request shapes, response models and HTTP methods. This supports verification that the API matches the intended design.
3. Role-based checks: the same action was attempted with different accounts (end user and admin). Expected outcomes: forbidden responses or hidden controls for unauthorised roles, and successful updates only for administrators where specified.
4. Data integrity: MongoDB Compass was used to confirm that users, tickets and notifications are stored as expected after key actions, such as a notification document exists after resolution)
5. Environment and configuration: Verification included running with and without optional services (SendGrid): without an API key, email is logged rather than sent, which confirms graceful behaviour for development demo
6. UI Consistency: validation included checking that login and dashboard views do not appear together incorrectly, that navigation matches the role, and that API error messages surface sensibly in the browser.

These activities continue specification-driven verification with scenario-based validation.

2.2. Testing

Testing was primarily manual and exploratory. Supported by API level checks with an emphasis on repeatable manual procedures.

Authentication and authorisation

- Log in with valid credentials; confirm token-based access to protected routes (/auth/me, /tickets).
- Log in with invalid credentials; confirm appropriate error handling.
- Attempt privileged actions (e.g. PATCH /tickets/{id}) as a non-administrator; confirm HTTP 403 and that the frontend does not expose admin-only controls to end users.
- Log in as administrator; confirm the ticket status can be updated, and the user list is accessible where required.

Ticket lifecycle

- Create a ticket with all required fields; confirm it appears in the list with status “open” (or equivalent initial state).
- Filter tickets by status; confirm filtering matches backend query parameters.
- As administrator, change status through the UI (e.g. open → in progress → resolved); confirm the table refreshes and status persists after refresh.
- Optionally add resolution notes when marking resolved; confirm they are accepted by the API and appear in email or notification content as designed.

Notifications

- After an administrator resolves a ticket, log in as the ticket creator; confirm the in-app notifications panel appears with an unread “resolved” message.
- Dismiss a single notification and “dismiss all”; confirm the panel hides when no unread items remain and that behaviour aligns with the mark-read API.

Email (if configured)

- With SendGrid configured, resolve a ticket and check the creator’s inbox.
- Without SendGrid, confirm the backend logs the intended email content so behaviour can still be verified during development.

Regression and edge cases

- Empty or invalid form submissions on the frontend (browser validation and API errors).
- Inactive user account, login should fail with a clear message if is_active is false.
- Large or unusual inputs (long titles/descriptions) to observe validation and stability.

Limitations of the current approach

- Manual testing is time-consuming and not automatically repeated on every change.
- There are no unit tests for individual functions or integration tests for full request flows in CI.
- Load and security testing (penetration testing, stress testing) were not involved in this project.

Possible improvements

- Add pytest tests for critical backend paths (auth, ticket update rules, notification creation).
- Add a small set of API integration tests (e.g. with httpx and Test Client) for role checks.

3. Critical appraisal

(suggested word count for Critical appraisal: 1000)

3.1. Evaluation of project outcomes

The project delivered a working helpdesk-style application with a clear separation between a Fast API backend and a lightweight frontend. The key outcomes align with typical IT support workflows: users can issue tickets, admins can progress and resolve them, and ticket creators can be informed through in-app notifications when they next log in.

Strength:

- Clear role model: End users, support staff, and admins are distinguished in both API and UI, with administrators holding privileged actions such as updating ticket status and managing user accounts.
- Traceability: ticket history records changes, which supports accountability- important in a real-time helpdesk system.
- Dual notifications: Email integration (SendGrid) plus persistent in-app notifications reduce reliance on a single channel and match the requirements that users see resolution feedback inside the application.
- Practical stack: Fast API, MongoDB, and JWT are widely used. The solution has been demonstrated to run locally.

Limitations:

- The frontend is intentionally simple. It is adequate for demonstration and small-scale use, but lacks the structure, testing and accessibility suitable for this project.
- There is no real-time collaboration (e.g., live ticket updates), file attachments are modelled but not fully productised, and scalability (caching) was not a primary goal.
- Automated tests are not a major aspect because manual and API checks were the priority.

3.2. Review of plans and any decisions

MongoDB over a relational database:

The document model fits evolving ticket fields and notification records without rigid migrations early on. The trade-off is less built-in relational integrity than SQL provides; careful application logic is required to keep references (e.g., user_id on notifications) consistent.

Administrator-only ticket updates:

A stricter rule was chosen; only administrators can change ticket status. That simplifies accountability but may not match every organisation where support staff also resolve tickets. The decision is defensible for a small team where IT leads approve closure.

Simple frontend vs Angular/React:

A minimal HTML/JS client reduced complexity and time to integrate with the API. The trade-off is more manual DOM work and fewer out-of-the-box patterns for state management and routing.

Notifications as stored documents:

In-app notifications persisted in the database rather than only being shown in-session. That supports the “see when you log in next” requirement but implies ongoing storage and eventual clean-up or archiving policies in a long-running system.

Optional SendGrid:

Falling back to console logging when email is not configured was a pragmatic decision for local development and marking, without blocking other features.

3.3. Evaluation of methodology

What worked well

- **API-first thinking:** Defining resources, auth, tickets, users, and notifications gave the frontend stable contracts.
- **FastAPI tooling:** Interactive docs, accelerated manual verification of payloads, and status codes.
- **Role-based manual testing:** Logging in as different users exposed authorisation gaps quickly.

What could be improved

- **Test-driven or test-supported development:** Adding a small pytest suite for auth and ticket rules would have reduced regression risk as features grew.
- **Formal requirements document:** A short living spec (user stories + acceptance criteria) would make “done” less subjective.
- **Version control discipline:** Commit messages and branches tied to features improve traceability for assessors in the future.
- **Time budgeting:** If time for the project's practical development had been better managed, then more time could have been spent debugging issues with Angular for the initial design, despite basic HTML being a suitable backup

The methodology suits a small academic project like this, but would require more formal processes such as CI, code review, and definition of done in a professional team environment.

3.4. Personal development

Technical skills

- **Backend engineering:** Designing REST endpoints, handling errors consistently, and using async database access with Motor.
- **Security awareness:** JWT flows, password hashing, and the difference between authentication (who you are) and authorisation (what you may do).
- **Data modelling:** Mapping domain concepts (ticket, user, notification) to documents and APIs.
- **Integration:** Connecting a browser client to an API, CORS, and environment-based configuration.

Professional skills

- **Debugging:** Interpreting HTTP status codes, API responses, and database state (e.g. via Compass) when behaviour did not match expectations.
- **Communication:** Explaining architecture, trade-offs, and test strategy in a report or demo is as important as writing code.

Reflection

Through this project, I've learned that the true measure of a feature isn't just getting it to run—it's about making it observable and testable. I've realised that a feature doesn't truly "count" until I can see it working from the end user's perspective, like verifying that a resolution notification appears after login.

I also came to appreciate how constraints—whether they involve tight deadlines, limited scope, or working solo—force me to be a better strategist. Choosing a simple frontend and a clear set of admin rules wasn't just a technical decision; it was a rational response to the reality of those limitations.

Looking ahead, I'm excited to level up my technical foundation. My goals include becoming comfortable with pytest and CI basics, mastering a mainstream frontend framework for handling more complex UIs, and diving deeper into the nuances of secure deployment, with a focus on HTTPS, secrets management, and OWASP-oriented security reviews.

References

Appendix A - Code Manifest

(Section not included in word count)

Filename	Contribution	Purpose	Path to file (in AT3 zip file)
Main.py	FastAPI application entry	Creates the app, registers routers, CORS, startup/shutdown DB hooks, health/root routes	helpdesk_backend/app/main.py
Database.py	Data access/MongoDB	Async Motor client, connection string from env, connect/close and shared db handle	helpdesk_backend/app/database.py
__init__.py	Package maker	Marks app as a Python package	helpdesk_backend/app/__init__.py
User.py	Domain model - users	User document/schema definitions for MongoDB	helpdesk_backend/app/models/user.py
Ticket.py	Domain model - tickets	Ticket document/schema definitions for MongoDB	helpdesk_backend/app/models/ticket.py
Security.py	Security utilities	Password hashing, JWT (or related auth crypto) helpers	helpdesk_backend/app/core/security.py
Email.py	Notifications -email	Sending or formatting email for notifications	helpdesk_backend/app/core/email.py
Auth.py	REST API Authentication	Login, tokens, registration, or auth-protected operations	helpdesk_backend/app/routers/auth.py
Tickets.py	REST API tickets	CRUD and business routes for helpdesk tickets	helpdesk_backend/app/routers/tickets.py
Users.py	REST API users	User management routes (list, update, status, etc.)	helpdesk_backend/app/routers/users.py
Dashboard.py	REST API dashboard	Aggregated stats or dashboard data for the UI	helpdesk_backend/app/routers/dashboard.py
Notifications.py	REST API notifications	Notification listing or triggers	helpdesk_backend/app/routers/notifications.py
__init__.py	Router package	Exposes router submodules	helpdesk_backend/app/routers/__init__.py
Reactive_user.py	Admin/maintenance utility	One-off script to reactivate a user by email in the database	helpdesk_backend/reactivate_user.py
Index.html	Frontend structure	Main page markup for the simple helpdesk UI	simple_frontend/index.html

App.js	Frontend behaviour	Fetches API, DOM updates, client-side logic	simple_frontend/app.js
Style.css	Frontend presentation	Layout, typography, and styling	simple_frontend/styles.css

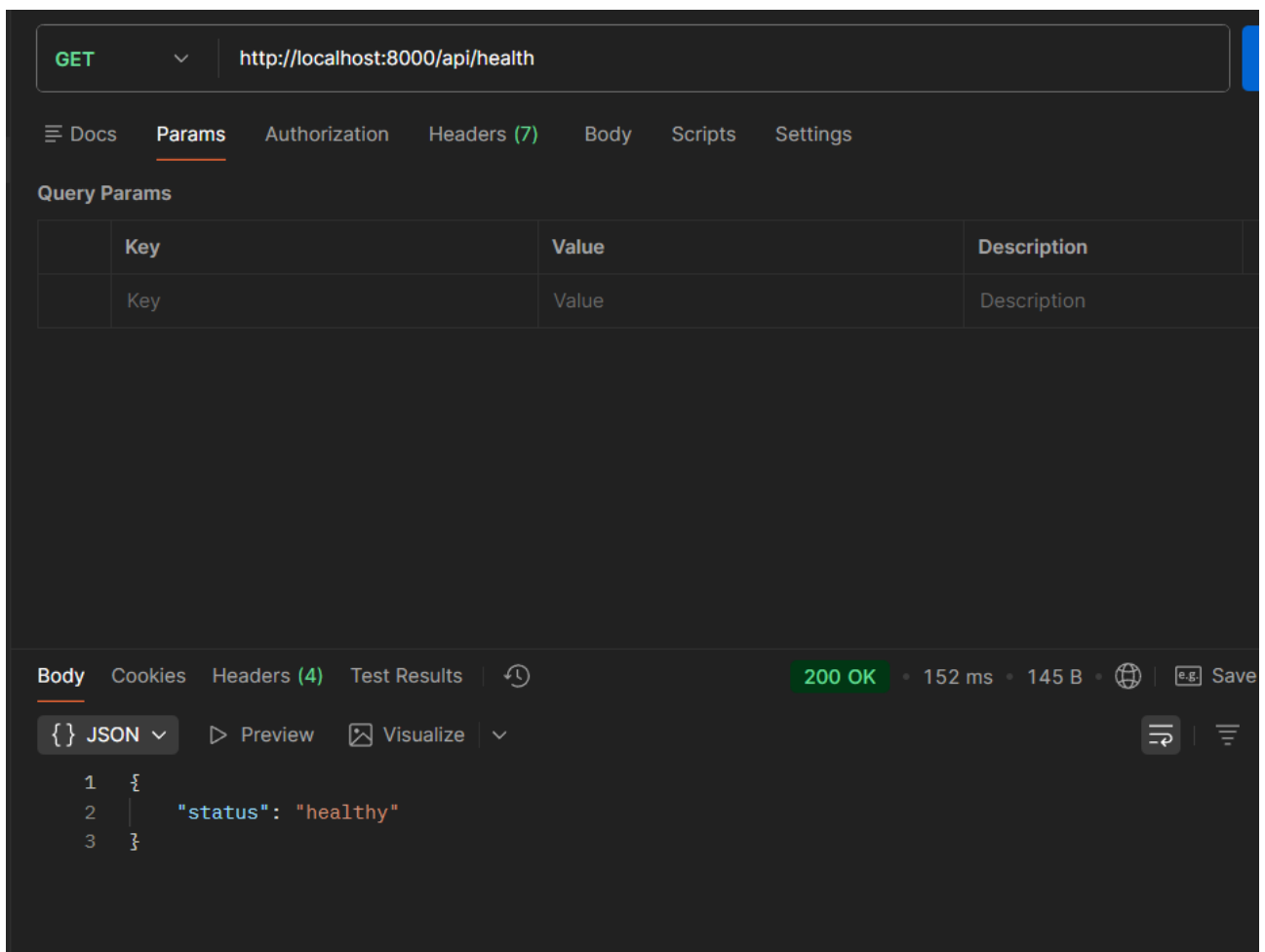
(Extend table as needed)

Additional appendices

(Section not included in suggested word count)

Note, appendices must not include narrative, discussion, analysis, or similar writing. They must only include additional items that evidence practice claimed, discussed or analysed within the main body of the report.

Postman API tests & screenshots



Screenshot above shows that the API is responding and 'Healthy'